

SQL Phase : 2

Section 1 — JOINS & CASE

Q1. Customer Order Report

Using appropriate JOINS, display:

- Customer Name
- Customer Segment
- Region
- Order Number
- Order Date
- Order Channel
- Order Status
- Final Order Amount

Sort by **Order Date descending**.

```
SET search_path TO ecommerce_order_delivery;
```

```
SELECT c.customer_name,  
       c.customer_segment,  
       r.region_zone AS region,  
       o.order_number,  
       o.order_date,  
       o.order_channel,  
       o.order_status,  
       o.final_order_amount
```

```

FROM fact_orders o
JOIN dim_customer c
    ON o.customer_id = c.customer_id
JOIN dim_region r
    ON c.region_id = r.region_id
ORDER BY o.order_date DESC;

```

	customer_name	customer_segment	region	order_number	order_date	order_channel	order_status	final_order_amount
1	Vivaan Iyer 242	Mass Affluent	West	ORD20250015052	2025-12-30	Website	Delivered	7047.91
2	Aarav Patil 761	Mass Affluent	West	ORD20250019931	2025-12-30	Mobile App	Delivered	43349.48
3	Riya Mehta 217	Affluent	West	ORD20250003737	2025-12-30	Mobile App	Delivered	44660.00
4	Rahul Shah 1760	Mass Affluent	West	ORD20250015571	2025-12-30	Website	Delivered	189645.22
5	Neha Mehta 952	Mass Affluent	West	ORD20250012353	2025-12-30	Website	Delivered	32830.57
6	Pooja Sharma 733	Mass Affluent	West	ORD20250017751	2025-12-30	Website	Delivered	60416.32
7	Kavya Kulkarni 1254	Mass Affluent	West	ORD20250001557	2025-12-30	Website	Delivered	228660.47

Q2. Order Item Sales Report

Create a report showing:

- Order Number
- Customer Name
- Product Name
- Category
- Seller Name
- Warehouse Name
- Quantity
- Unit Price

- Item Discount
- Item Total Amount

Use the required JOINS across the order, item, product, category, seller and warehouse tables.

```
SET search_path TO ecommerce_order_delivery;
```

```
SELECT
```

```
    o.order_number,  
    c.customer_name,  
    p.product_name,  
    cat.category_name AS category,  
    s.seller_name,  
    w.warehouse_name,  
    foi.quantity,  
    foi.unit_price,  
    foi.item_discount,  
    foi.item_total_amount
```

```
FROM fact_order_items foi
```

```
JOIN fact_orders o  
    ON foi.order_id = o.order_id
```

```
JOIN dim_customer c  
    ON o.customer_id = c.customer_id
```

```
JOIN dim_product p  
    ON foi.product_id = p.product_id
```

```
JOIN dim_category cat  
    ON p.category_id = cat.category_id
```

```
JOIN dim_seller s  
    ON foi.seller_id = s.seller_id
```

```
JOIN dim_warehouse w
ON foi.warehouse_id = w.warehouse_id;
```

The screenshot shows a SQL IDE interface. On the left is a sidebar with a tree view of database objects including tables like dim_calendar, dim_category, dim_customer, dim_delivery_partner, dim_product, dim_region, dim_seller, dim_warehouse, fact_customer_reviews, fact_inventory_snapshot, fact_order_items, fact_orders, fact_payments, fact_returns, fact_seller_performance, and fact_shipments. The main editor displays a SQL query starting with 'SET search_path TO ecommerce_order_delivery;' followed by a SELECT statement joining fact_order_items (foi) and fact_orders (o) on foi.order_id = o.order_id. The query selects various fields including order_number, customer_name, product_name, category, seller_name, warehouse_name, quantity, unit_price, item_discount, and item_total_amount. Below the editor, a 'Data Output' pane shows the first five rows of the query results.

	order_number	customer_name	product_name	category	seller_name	warehouse_name	quantity	unit_price	item_discount	item_total_amount
1	ORD20240000001	Meera Joshi 18	iPhone Series Smartphone 4	Mobiles & Smartphones	Tech World 11	Jaipur Regional Hub 5	1	58110.88		
2	ORD20240000002	Isha Shah 35	Pixel Series Smartphone 15	Mobiles & Smartphones	Mobile Planet 18	Lucknow Distribution Hub ...	1	41275.80		
3	ORD20240000003	Neha Mehta 52	Galaxy A Series Smartphone ...	Mobiles & Smartphones	Digital Store 25	Ahmedabad Distribution H...	1	68724.92		
4	ORD20240000004	Ananya Kulkarni 69	Redmi Note Smartphone 7	Mobiles & Smartphones	Smart Retail 32	Chennai Fulfilment Center 8	1	17974.04		
5	ORD20240000005	Kunal Patil 86	Smart LED TV 36	Televisions	Fashion Hub 4	Mumbai Fulfilment Center 1	1	120899.04		

Q3. Product Sales Performance

For every product calculate:

- Product Name
- Category
- Number of Orders
- Units Sold
- Gross Sales
- Total Discount
- Net Sales

Sort by **Net Sales descending**.

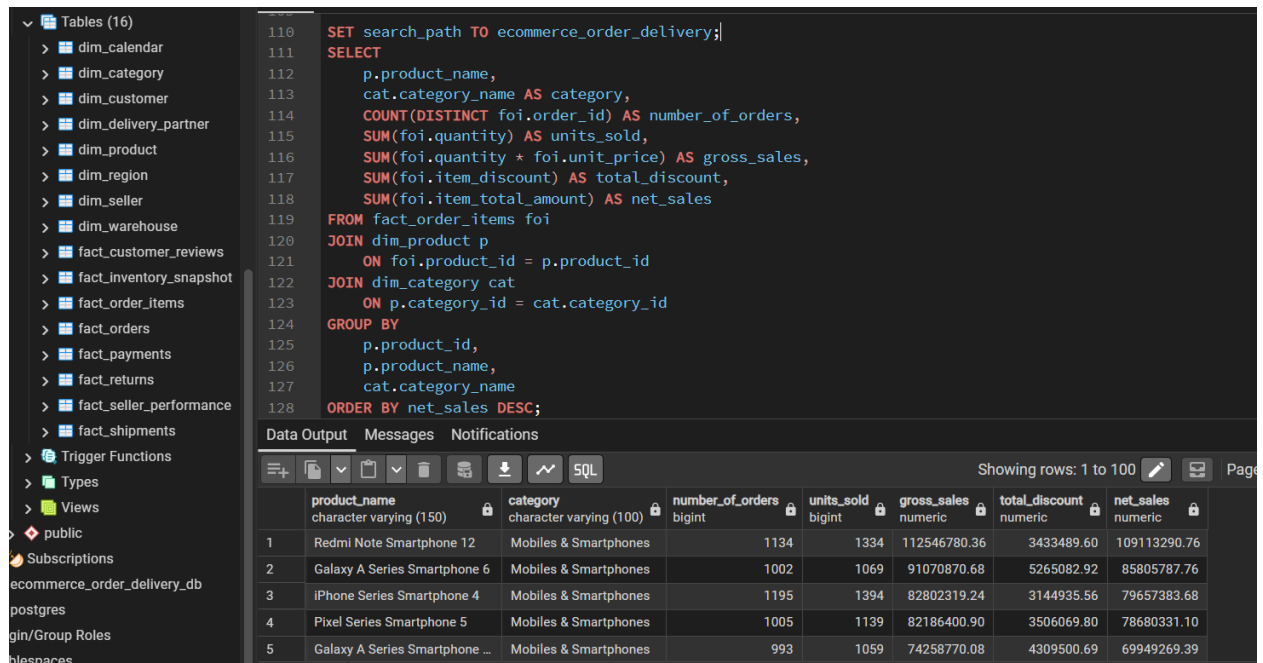
```
SET search_path TO ecommerce_order_delivery;
```

```
SELECT
```

```

p.product_name,
cat.category_name AS category,
COUNT(DISTINCT foi.order_id) AS number_of_orders,
SUM(foi.quantity) AS units_sold,
SUM(foi.quantity * foi.unit_price) AS gross_sales,
SUM(foi.item_discount) AS total_discount,
SUM(foi.item_total_amount) AS net_sales
FROM fact_order_items foi
JOIN dim_product p
    ON foi.product_id = p.product_id
JOIN dim_category cat
    ON p.category_id = cat.category_id
GROUP BY
    p.product_id,
    p.product_name,
    cat.category_name
ORDER BY net_sales DESC;

```



The screenshot shows a database IDE interface. On the left is a sidebar with a tree view of database objects: Tables (16), Trigger Functions, Types, Views, public, Subscriptions, ecommerce_order_delivery_db, postgres, gin/Group Roles, and tablespaces. The main area displays a SQL query in a dark-themed editor. The query is identical to the one in the first block. Below the editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 8 columns: product_name, category, number_of_orders, units_sold, gross_sales, total_discount, and net_sales. The table contains 5 rows of data. The status bar at the bottom right indicates 'Showing rows: 1 to 100'.

	product_name character varying (150)	category character varying (100)	number_of_orders bigint	units_sold bigint	gross_sales numeric	total_discount numeric	net_sales numeric
1	Redmi Note Smartphone 12	Mobiles & Smartphones	1134	1334	112546780.36	3433489.60	109113290.76
2	Galaxy A Series Smartphone 6	Mobiles & Smartphones	1002	1069	91070870.68	5265082.92	85805787.76
3	iPhone Series Smartphone 4	Mobiles & Smartphones	1195	1394	82802319.24	3144935.56	79657383.68
4	Pixel Series Smartphone 5	Mobiles & Smartphones	1005	1139	82186400.90	3506069.80	78680331.10
5	Galaxy A Series Smartphone ...	Mobiles & Smartphones	993	1059	74258770.08	4309500.69	69949269.39

Q4. Customer Spending Classification

Using **CASE**, classify customers based on their **total final order amount**:

Total Spending	Customer Type
< ₹10,000	Low Value
₹10,000–₹50,000	Regular
₹50,000–₹1,00,000	High Value
> ₹1,00,000	VIP

Display:

- Customer Name
- Customer Segment
- Total Orders
- Total Spending
- Spending Category

```
SET search_path TO ecommerce_order_delivery;

SELECT
    c.customer_name,
    c.customer_segment,
    COUNT(o.order_id) AS total_orders,
    SUM(o.final_order_amount) AS total_spending,

    CASE
        WHEN SUM(o.final_order_amount) < 10000
            THEN 'Low Value'

        WHEN SUM(o.final_order_amount) BETWEEN 10000 AND 50000
            THEN 'Regular'

        WHEN SUM(o.final_order_amount) > 50000
            THEN 'High Value'

        WHEN SUM(o.final_order_amount) > 100000
            THEN 'VIP'
    END AS spending_category
FROM customer c
LEFT JOIN order o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.customer_segment;
```

```

        WHEN SUM(o.final_order_amount) > 50000
            AND SUM(o.final_order_amount) <= 100000
        THEN 'High Value'

        WHEN SUM(o.final_order_amount) > 100000
        THEN 'VIP'
    END AS spending_category

FROM fact_orders o
JOIN dim_customer c
    ON o.customer_id = c.customer_id

GROUP BY
    c.customer_id,
    c.customer_name,
    c.customer_segment;

```

Query Editor Interface:

- 1.3 Sequences
- Tables (16)
 - dim_calendar
 - dim_category
 - dim_customer
 - dim_delivery_partner
 - dim_product
 - dim_region
 - dim_seller
 - dim_warehouse
 - fact_customer_reviews
 - fact_inventory_snapshot
 - fact_order_items
 - fact_orders
 - fact_payments
 - fact_returns
 - fact_seller_performance
 - fact_shipments
- Trigger Functions
- Types
- Views
- public
- Subscriptions
- ecommerce_order_delivery_db
- postgres
- gin/Group Roles
- namespaces

Query

```

SELECT
    c.customer_name,
    c.customer_segment,
    COUNT(o.order_id) AS total_orders,
    SUM(o.final_order_amount) AS total_spending,

    CASE
        WHEN SUM(o.final_order_amount) < 10000
            THEN 'Low Value'

        WHEN SUM(o.final_order_amount) BETWEEN 10000 AND 50000
            THEN 'Regular'

        WHEN SUM(o.final_order_amount) > 50000
            AND SUM(o.final_order_amount) <= 100000
            THEN 'High Value'

        WHEN SUM(o.final_order_amount) > 100000
            THEN 'VIP'
    
```

Query History

Data Output

	customer_name character varying (100)	customer_segment character varying (50)	total_orders bigint	total_spending numeric	spending_category text
1	Ananya Singh 1489	Mass Market	18	1707422.40	VIP
2	Neha Mehta 652	Mass Affluent	8	376774.40	VIP
3	Pooja Joshi 273	Mass Market	7	336826.95	VIP
4	Sneha More 51	Mass Affluent	7	214632.32	VIP
5	Rahul Khan 1560	Mass Affluent	18	2765559.66	VIP

Showing rows: 1 to 5

Q5. Order Value Classification

Using `CASE`, classify orders based on `final_order_amount`:

Order Amount	Category
< ₹1,000	Small
₹1,000–₹5,000	Medium
₹5,000–₹20,000	Large
> ₹20,000	Premium

Display:

- Order Number
- Customer
- Order Date
- Final Order Amount
- Order Value Category

```
SET search_path TO ecommerce_order_delivery;
```

```
SELECT
```

```
    o.order_number,  
    c.customer_name AS customer,  
    o.order_date,  
    o.final_order_amount,
```

```
    CASE
```

```
        WHEN o.final_order_amount < 1000  
            THEN 'Small'
```

```
        WHEN o.final_order_amount BETWEEN 1000 AND 5000  
            THEN 'Medium'
```



```

        WHEN o.final_order_amount > 5000
            AND o.final_order_amount <= 20000
        THEN 'Large'

        WHEN o.final_order_amount > 20000
        THEN 'Premium'
    END AS order_value_category

FROM fact_orders o
JOIN dim_customer c
    ON o.customer_id = c.customer_id;

```

The screenshot shows a database management interface. On the left is a sidebar with a tree view of database objects: Sequences, Tables (16), Trigger Functions, Types, Views, and public. Under Tables, several tables are listed, including dim_calendar, dim_category, dim_customer, dim_delivery_partner, dim_product, dim_region, dim_seller, dim_warehouse, fact_customer_reviews, fact_inventory_snapshot, fact_order_items, fact_orders, fact_payments, fact_returns, fact_seller_performance, and fact_shipments. The main area is divided into two panes. The top pane, titled 'Query', shows a SQL query that sets a search path and selects columns from fact_orders and dim_customer, including a CASE statement for order_value_category. The bottom pane, titled 'Data Output', shows the results of the query as a table with 5 rows and 6 columns: order_number, customer, order_date, final_order_amount, and order_value_category. The results show five orders with their respective customer names, dates, amounts, and categories (Premium or Large).

	order_number	customer	order_date	final_order_amount	order_value_category
1	ORD20240000001	Meera Joshi 18	2024-01-08	50531.20	Premium
2	ORD20240000002	Isha Shah 35	2024-01-15	35892.00	Premium
3	ORD20240000003	Neha Mehta 52	2024-01-22	59760.80	Premium
4	ORD20240000004	Ananya Kulkarni 69	2024-01-29	15629.60	Large
5	ORD20240000005	Kunal Patil 86	2024-02-05	105129.60	Premium



Section 2 — Date Functions & Analysis

Q6. Monthly Sales Analysis

Using `TO_CHAR()`, `EXTRACT()` or `DATE_TRUNC()`, calculate monthly:

- Year
- Month
- Number of Orders
- Units Sold
- Gross Sales
- Final Sales Amount

Sort chronologically.

```
SET search_path TO ecommerce_order_delivery;

SELECT
    EXTRACT(YEAR FROM o.order_date) AS year,
    EXTRACT(MONTH FROM o.order_date) AS month,
    COUNT(DISTINCT o.order_id) AS number_of_orders,
    SUM(foi.quantity) AS units_sold,
    SUM(foi.quantity * foi.unit_price) AS gross_sales,
    SUM(o.final_order_amount) AS final_sales_amount
FROM fact_orders o
JOIN fact_order_items foi
    ON o.order_id = foi.order_id
GROUP BY
    EXTRACT(YEAR FROM o.order_date),
    EXTRACT(MONTH FROM o.order_date)
ORDER BY
    year,
    month;
```

1.3 Sequences

Tables (16)

dim_calendar
dim_category
dim_customer
dim_delivery_partner
dim_product
dim_region
dim_seller
dim_warehouse
fact_customer_reviews
fact_inventory_snapshot
fact_order_items
fact_orders
fact_payments
fact_returns
fact_seller_performance
fact_shipments
Trigger Functions
Types
Views
public
Subscriptions
commerce_order_delivery_db
postgres
in/Group Roles
namespaces

Query
Query History

257 SET search_path TO ecommerce_order_delivery;
258
259 SELECT
260 EXTRACT(YEAR FROM o.order_date) AS year,
261 EXTRACT(MONTH FROM o.order_date) AS month,
262 COUNT(DISTINCT o.order_id) AS number_of_orders,
263 SUM(foi.quantity) AS units_sold,
264 SUM(foi.quantity * foi.unit_price) AS gross_sales,
265 SUM(o.final_order_amount) AS final_sales_amount
266 FROM fact_orders o
267 JOIN fact_order_items foi
268 ON o.order_id = foi.order_id
269 GROUP BY
270 EXTRACT(YEAR FROM o.order_date),
271 EXTRACT(MONTH FROM o.order_date)
272 ORDER BY
273 year,
274 month;

Data Output
Messages
Notifications

Showing rows: 1 to 24

	year numeric	month numeric	number_of_orders bigint	units_sold bigint	gross_sales numeric	final_sales_amount numeric
1	2024	1	851	1669	74932368.38	156994094.03
2	2024	2	795	1574	70024408.50	146304365.47
3	2024	3	851	1667	75683491.46	157912022.12
4	2024	4	824	1617	73146828.06	151811847.07
5	2024	5	850	1670	75941026.90	157572723.77
6	2024	6	823	1627	72297144.03	150696652.31

Q7. Weekend vs Weekday Sales

Using the order date and `dim_calendar`, compare:

- Weekday Orders
- Weekend Orders
- Total Sales
- Average Order Value

Display the results as:

Day Type | Orders | Total Sales | Average Order Value

```

SELECT
CASE
    WHEN dc.is_weekend = TRUE THEN 'Weekend'
    ELSE 'Weekday'

```

```
END AS day_type,

COUNT(o.order_id) AS orders,
SUM(o.final_order_amount) AS total_sales,
AVG(o.final_order_amount) AS average_order_value

FROM fact_orders o

JOIN dim_calendar dc
  ON o.order_date = dc.calendar_date

GROUP BY
  CASE
    WHEN dc.is_weekend = TRUE THEN 'Weekend'
    ELSE 'Weekday'
  END

ORDER BY
  day_type;
```

The screenshot shows a SQL IDE interface. On the left is a sidebar with a tree view of database objects: Sequences, Tables (16), Trigger Functions, Types, Views, public, Subscriptions, ecommerce_order_delivery_db, postgres, and Group Roles. The 'Tables (16)' folder is expanded, and 'dim_calendar' is selected. The main area is split into 'Query' and 'Query History' tabs. The 'Query' tab shows a SQL query (lines 295-317) that calculates order statistics grouped by day type. Below the query is a 'Data Output' tab showing the results of the query. The results are displayed in a table with 5 columns: day_type, orders, total_sales, and average_order_value. The table shows two rows: 'Weekday' and 'Weekend'.

```

295 SELECT
296     CASE
297         WHEN dc.is_weekend = TRUE THEN 'Weekend'
298         ELSE 'Weekday'
299     END AS day_type,
300
301     COUNT(o.order_id) AS orders,
302     SUM(o.final_order_amount) AS total_sales,
303     AVG(o.final_order_amount) AS average_order_value
304
305 FROM fact_orders o
306
307 JOIN dim_calendar dc
308     ON o.order_date = dc.calendar_date
309
310 GROUP BY
311     CASE
312         WHEN dc.is_weekend = TRUE THEN 'Weekend'
313         ELSE 'Weekday'
314     END
315
316 ORDER BY
317     day_type;

```

	day_type	orders	total_sales	average_order_value
1	Weekday	14320	1234173866.91	86185.325901536313
2	Weekend	5680	498477987.43	87760.209054577465

Q8. Delivery Delay Analysis

Using shipment dates, calculate the number of days between:

Estimated Delivery Date → Actual Delivery Date

Classify delivery performance using **CASE** :

- Delivered Early
- On Time
- 1–3 Days Late
- 4–7 Days Late
- More Than 7 Days Late

Find the number of shipments in each category.

```

SELECT
    CASE
        WHEN actual_delivery_date < estimated_delivery_date
            THEN 'Delivered Early'

        WHEN actual_delivery_date = estimated_delivery_date
            THEN 'On Time'

        WHEN actual_delivery_date > estimated_delivery_date
            AND actual_delivery_date - estimated_delivery_date
te BETWEEN 1 AND 3
            THEN '1-3 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date B
ETWEEN 4 AND 7
            THEN '4-7 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date >
7
            THEN 'More Than 7 Days Late'
    END AS delivery_performance,

    COUNT(shipment_id) AS number_of_shipments

FROM fact_shipments

WHERE actual_delivery_date IS NOT NULL

GROUP BY
    CASE
        WHEN actual_delivery_date < estimated_delivery_date
            THEN 'Delivered Early'

        WHEN actual_delivery_date = estimated_delivery_date
            THEN 'On Time'
    
```

```

        WHEN actual_delivery_date > estimated_delivery_date
             AND actual_delivery_date - estimated_delivery_date
te BETWEEN 1 AND 3
             THEN '1-3 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date B
ETWEEN 4 AND 7
             THEN '4-7 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date >
7
             THEN 'More Than 7 Days Late'

END;

```

Query Editor Interface:

- Left Panel (Database Explorer):**
 - 1.3 Sequences
 - Tables (16)
 - dim_calendar
 - dim_category
 - dim_customer
 - dim_delivery_partner
 - dim_product
 - dim_region
 - dim_seller
 - dim_warehouse
 - fact_customer_reviews
 - fact_inventory_snapshot
 - fact_order_items
 - fact_orders
 - fact_payments
 - fact_returns
 - fact_seller_performance
 - fact_shipments
 - Trigger Functions
 - Types
 - Views
 - public
 - Subscriptions
 - commerce_order_delivery_db
 - stgres
 - Group Roles
 - spaces
- Query Editor:**

```

338 SELECT
339 CASE
340     WHEN actual_delivery_date < estimated_delivery_date
341     THEN 'Delivered Early'
342
343     WHEN actual_delivery_date = estimated_delivery_date
344     THEN 'On Time'
345
346     WHEN actual_delivery_date > estimated_delivery_date
347     AND actual_delivery_date - estimated_delivery_date BETWEEN 1 AND 3
348     THEN '1-3 Days Late'
349
350     WHEN actual_delivery_date - estimated_delivery_date BETWEEN 4 AND 7
351     THEN '4-7 Days Late'
352
353     WHEN actual_delivery_date - estimated_delivery_date > 7
354     THEN 'More Than 7 Days Late'
355 END AS delivery_performance,
356

```
- Data Output:**

	delivery_performance text	number_of_shipments bigint
1	On Time	5687
2	4-7 Days Late	400
3	1-3 Days Late	1800
4	Delivered Early	9113
5	More Than 7 Days Late	200

Q9. Return Analysis by Month

Using `return_date`, calculate monthly:

- Number of Returns
- Units Returned
- Refund Amount
- Average Refund Amount

Also identify the month with the **highest refund amount**.

```
SELECT
    EXTRACT(YEAR FROM return_date) AS year,
    EXTRACT(MONTH FROM return_date) AS month,
    COUNT(return_id) AS number_of_returns,
    SUM(quantity_returned) AS units_returned,
    SUM(refund_amount) AS refund_amount,
    AVG(refund_amount) AS average_refund_amount
FROM fact_returns
GROUP BY
    EXTRACT(YEAR FROM return_date),
    EXTRACT(MONTH FROM return_date)
ORDER BY
    year,
    month;
```


The screenshot shows a SQL IDE interface. On the left is a tree view of database objects including Procedures, Sequences, and Tables (16). The 'Tables' folder is expanded, showing various dimension and fact tables. The main pane displays a SQL query (lines 398-411) that extracts year and month from 'return_date' and calculates aggregate statistics for 'fact_returns'.

Query:

```

398 SELECT
399     EXTRACT(YEAR FROM return_date) AS year,
400     EXTRACT(MONTH FROM return_date) AS month,
401     COUNT(return_id) AS number_of_returns,
402     SUM(quantity_returned) AS units_returned,
403     SUM(refund_amount) AS refund_amount,
404     AVG(refund_amount) AS average_refund_amount
405 FROM fact_returns
406 GROUP BY
407     EXTRACT(YEAR FROM return_date),
408     EXTRACT(MONTH FROM return_date)
409 ORDER BY
410     year,
411     month;

```

Below the query editor, the 'Data Output' tab is active, showing a table with 9 rows of results. The table has 7 columns: year, month, number_of_returns, units_returned, refund_amount, and average_refund_amount. The data shows results for each month of the year 2024.

	year numeric	month numeric	number_of_returns bigint	units_returned bigint	refund_amount numeric	average_refund_amount numeric
1	2024	1	62	62	2787417.47	44958.346290322581
2	2024	2	113	113	5832336.91	51613.600973451327
3	2024	3	133	133	6627002.79	49827.088646616541
4	2024	4	124	124	6091598.85	49125.797177419355
5	2024	5	110	110	5484535.60	49859.414545454545
6	2024	6	133	133	5992331.01	45055.120375939850
7	2024	7	138	138	5450447.91	39495.999347826087
8	2024	8	117	117	4688424.71	40072.006068376068
9	2024	9	128	128	5331901.72	41655.482187500000

Section 3 — CTE

Q10. Customer Sales Summary Using CTE

Create a CTE that calculates for every customer:

- Total Orders
- Total Items Purchased
- Total Spending
- Average Order Value

Return customers whose spending is **above the overall average customer spending**.

```

SET search_path TO ecommerce_order_delivery;

WITH customer_summary AS (
    SELECT
        c.customer_id,
        c.customer_name,
        COUNT(DISTINCT o.order_id) AS total_orders,
        SUM(foi.quantity) AS total_items_purchased,
        SUM(o.final_order_amount) AS total_spending,
        AVG(o.final_order_amount) AS average_order_value
    FROM dim_customer c
    JOIN fact_orders o
        ON c.customer_id = o.customer_id
    JOIN fact_order_items foi
        ON o.order_id = foi.order_id
    GROUP BY
        c.customer_id,
        c.customer_name
)

SELECT
    customer_name,
    total_orders,
    total_items_purchased,
    total_spending,
    average_order_value
FROM customer_summary
WHERE total_spending > (
    SELECT AVG(total_spending)
    FROM customer_summary
)
ORDER BY total_spending DESC;

```

Schemas (2)

ecommerce_order_delivery

Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (16)

dim_calendar
dim_category
dim_customer
dim_delivery_partner
dim_product
dim_region
dim_seller
dim_warehouse
fact_customer_reviews
fact_inventory_snapshot

Query
Query History

```

433 WITH customer_summary AS (
434     SELECT
435         c.customer_id,
436         c.customer_name,
437         COUNT(DISTINCT o.order_id) AS total_orders,
438         SUM(foi.quantity) AS total_items_purchased,
439         SUM(o.final_order_amount) AS total_spending,
440         AVG(o.final_order_amount) AS average_order_value
441     FROM dim_customer c
442     JOIN fact_orders o
443         ON c.customer_id = o.customer_id
444     JOIN fact_order_items foi
445         ON o.order_id = foi.order_id
446     GROUP BY
447         c.customer_id,
448         c.customer_name

```

Data Output
Messages
Notifications

Showing rows

	customer_name character varying (100)	totalOrders bigint	totalItemsPurchased bigint	totalSpending numeric	average_order_value numeric
1	Aditi Sharma 2428	28	112	35812646.16	426341.025714285714
2	Meera Joshi 2658	29	145	34262420.80	295365.696551724138
3	Kavya Kulkarni 2514	28	112	34016934.66	404963.507857142857
4	Kavya Singh 2314	29	116	29398415.88	337912.826206896552
5	Meera Nair 2858	28	140	29034028.00	259232.392857142857
6	Meera Joshi 2358	29	145	27737281.60	239114.496551724138
7	Aditi Joshi 2628	29	116	26810502.42	308166.694482758621
8	Meera Nair 2558	29	145	26455962.40	229162.469655172413

Q11. Category Performance Using CTE

Create a CTE that calculates for every category:

- Number of Orders
- Units Sold
- Gross Sales
- Net Sales
- Return Quantity

Identify categories where:

Return Quantity > 5% of Units Sold

```
SET search_path TO ecommerce_order_delivery;
```

```

WITH category_performance AS (
    SELECT
        cat.category_id,
        cat.category_name,

        COUNT(DISTINCT foi.order_id) AS number_of_orders,

        SUM(foi.quantity) AS units_sold,

        SUM(foi.quantity * foi.unit_price) AS gross_sales,

        SUM(foi.item_total_amount) AS net_sales,

        COALESCE(SUM(fr.quantity_returned), 0) AS return_quantity

    FROM dim_category cat

    JOIN dim_product p
        ON cat.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    LEFT JOIN fact_returns fr
        ON foi.order_item_id = fr.order_item_id

    GROUP BY
        cat.category_id,
        cat.category_name
)

SELECT
    category_name,
    number_of_orders,
    units_sold,

```

```

    gross_sales,
    net_sales,
    return_quantity
FROM category_performance
WHERE return_quantity > units_sold * 0.05
ORDER BY return_quantity DESC;

```

485	WITH category_performance AS (
486	SELECT
487	cat.category_id,
488	cat.category_name,
489	
490	COUNT(DISTINCT foi.order_id) AS number_of_orders,
491	
492	SUM(foi.quantity) AS units_sold,
493	
494	SUM(foi.quantity * foi.unit_price) AS gross_sales,
495	
496	SUM(foi.item_total_amount) AS net_sales,
497	
498	COALESCE(SUM(fr.quantity_returned), 0) AS return_quantity

	category_name	number_of_orders	units_sold	gross_sales	net_sales	return_quantity
	character varying (100)	bigint	bigint	numeric	numeric	bigint
1	Mobiles & Smartphones	11000	18600	871322266.25	834364501.10	1050
2	Laptops	3600	4887	423237573.44	406482157.77	473
3	Televisions	3600	4888	372706069.76	357801110.35	470
4	Audio	2331	2824	12687075.20	12136753.58	273
5	Kitchen Appliances	2000	2600	25519852.98	24559188.64	234
6	Home Appliances	2001	2602	34346774.81	33039586.46	233
7	Furniture	1299	1499	55975038.96	54099700.44	117
8	Beauty & Personal Care	200	300	697452.00	686544.00	100
9	Women Fashion	400	400	2919146.00	2789102.00	50

Q12. Seller Performance Using CTE

Using `fact_seller_performance`, create a CTE to calculate for every seller:

- Total Orders
- Units Sold
- Gross Sales
- Returns

- Net Sales
- Average Rating

Then identify sellers with:

- Net Sales above average
- Rating above 4.0

```
SET search_path TO ecommerce_order_delivery;

WITH seller_summary AS (
    SELECT
        s.seller_id,
        s.seller_name,
        SUM(fsp.orders_count) AS total_orders,
        SUM(fsp.units_sold) AS units_sold,
        SUM(fsp.gross_sales_amount) AS gross_sales,
        SUM(fsp.returns_count) AS returns,
        SUM(fsp.net_sales_amount) AS net_sales,
        AVG(fsp.rating_average) AS average_rating
    FROM fact_seller_performance fsp
    JOIN dim_seller s
        ON fsp.seller_id = s.seller_id
    GROUP BY
        s.seller_id,
        s.seller_name
)

SELECT
    seller_name,
    total_orders,
    units_sold,
    gross_sales,
    returns,
    net_sales,
```

```

        average_rating
FROM seller_summary
WHERE net_sales > (
    SELECT AVG(net_sales)
    FROM seller_summary
)
AND average_rating > 4.0
ORDER BY net_sales DESC;

```

1.3 Sequences

Tables (16)

dim_calendar

dim_category

dim_customer

dim_delivery_partner

dim_product

dim_region

dim_seller

dim_warehouse

fact_customer_reviews

fact_inventory_snapsh

fact_order_items

fact_orders

fact_payments

fact_returns

fact_seller_performanc

fact_shipments

Trigger Functions

Types

Views

public

Subscriptions

ecommerce_order_delivery_db

postgres

login/Group Roles

Query

Query History

```
567 SELECT
568     seller_name,
569     total_orders,
570     units_sold,
571     gross_sales,
572     returns,
573     net_sales,
574     average_rating
575 FROM seller_summary
576 WHERE net_sales > (
577     SELECT round(AVG(net_sales),3)
578     FROM seller_summary
579 )
580 AND average_rating > 4.0
581 ORDER BY net_sales DESC;
582
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 7

	seller_name character varying (100)	total_orders bigint	units_sold bigint	gross_sales numeric	returns bigint	net_sales numeric	average_rating numeric
1	Daily Needs 6	12070	24636	66119460.00	451	63474681.60	4.4993160054719562
2	Value Mart 10	6942	13640	57826860.00	60	55513785.60	4.5004103967168263
3	Digital Store 5	11993	24461	57146645.00	440	54860779.20	4.5002735978112175
4	Urban Lifestyle 9	6935	14377	52823895.00	62	50710939.20	4.4989056087551300
5	Fashion Hub 4	12114	24704	49085928.00	451	47122490.88	4.5012311901504788
6	Mobile Planet 8	6964	13682	48079456.00	60	46156277.76	4.4974008207934337
7	Mobile Planet 18	6974	14452	45568116.00	62	43745391.36	4.4989056087551300

Q13. Delivery Partner Performance

Using a CTE, calculate for every delivery partner:

- Total Shipments
- Delivered Shipments

- Delayed Shipments
- Average Delivery Delay
- Shipping Cost

Calculate:

On-Time Delivery %

Identify the best-performing delivery partner.

```
WITH partner_summary AS (
    SELECT
        dp.delivery_partner_id,
        dp.partner_name,

        COUNT(fs.shipment_id) AS total_shipments,

        COUNT(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date <= fs.estimated_delivery_date
            THEN 1
            END
        ) AS delivered_shipments,

        COUNT(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date > fs.estimated_delivery_date
            THEN 1
            END
        ) AS delayed_shipments,
```



```

        AVG(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                THEN fs.actual_delivery_date - fs.estimated_d
elivery_date
            END
        ) AS average_delivery_delay,

        SUM(fs.shipping_cost) AS shipping_cost

FROM dim_delivery_partner dp

JOIN fact_shipments fs
    ON dp.delivery_partner_id = fs.delivery_partner_id

GROUP BY
    dp.delivery_partner_id,
    dp.partner_name
)

SELECT
    partner_name,
    total_shipments,
    delivered_shipments,
    delayed_shipments,
    average_delivery_delay,
    shipping_cost,

    ROUND(
        delivered_shipments * 100.0 / NULLIF(total_shipments,
0),
        2
    ) AS on_time_delivery_percentage

FROM partner_summary

```

ORDER BY

on_time_delivery_percentage DESC;

Query	Query History
603	WITH partner_summary AS (
604	SELECT
605	dp.delivery_partner_id,
606	dp.partner_name,
607	
608	COUNT(fs.shipment_id) AS total_shipments,
609	
610	COUNT(
611	CASE
612	WHEN fs.actual_delivery_date IS NOT NULL
613	AND fs.actual_delivery_date <= fs.estimated_delivery_date
614	THEN 1
615	END
616	) AS delivered_shipments,
617	
618	COUNT(
619	CASE

partner_name	total_shipments	delivered_shipments	delayed_shipments	average_delivery_delay	shipping_cost	on_time_delivery_percentage
character varying (100)	bigint	bigint	bigint	numeric	numeric	numeric
1 Ecom Express	3000	3000	0	0.00000000000000000000	1192173.40	100.00
2 Delhivery	3800	3800	0	-0.73684210526315789474	1488069.05	100.00
3 BlueDart Logistics	4000	4000	0	-0.92825000000000000000	1450785.20	100.00
4 FastTrack Logistics	4800	2800	0	-0.92857142857142857143	1702100.60	58.33
5 XpressBees	2200	1200	1000	0.90909090909090909091	909562.50	54.55
6 Shadowfax	1000	0	1000	2.80000000000000000000	407304.30	0.00
7 Rational Courier	400	0	400	7.50000000000000000000	168345.00	0.00

Q14. Product Return Analysis

Using a CTE:

1. Calculate total quantity sold for every product.
2. Calculate total quantity returned.
3. Calculate return rate.

Return the **Top 20 products with the highest return rate**, considering only products with at least **20 units sold**.

```
WITH product_return_summary AS (
    SELECT
        p.product_id,
```

```

        p.product_name,

        SUM(foi.quantity) AS total_quantity_sold,

        COALESCE(SUM(fr.quantity_returned), 0) AS total_quant
ity_returned

    FROM dim_product p

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    LEFT JOIN fact_returns fr
        ON foi.order_item_id = fr.order_item_id

    GROUP BY
        p.product_id,
        p.product_name
)

SELECT
    product_id,
    product_name,
    total_quantity_sold,
    total_quantity_returned,

    ROUND(
        total_quantity_returned * 100.0
        / NULLIF(total_quantity_sold, 0),
        2
    ) AS return_rate

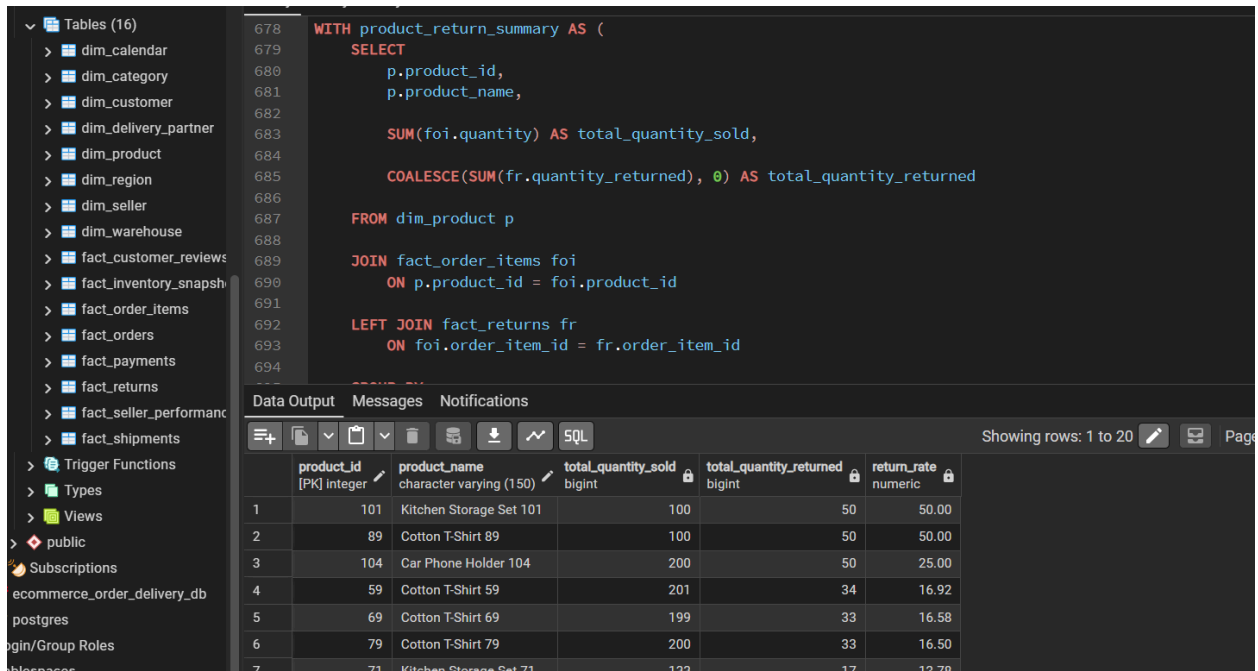
FROM product_return_summary

WHERE total_quantity_sold >= 20

```

ORDER BY return_rate DESC

LIMIT 20;



The screenshot shows a database IDE with a table list on the left and a SQL editor on the right. The SQL editor contains a query that creates a view named 'product_return_summary'. The query uses a CTE to calculate the total quantity sold and returned for each product, and then calculates the return rate. The results are displayed in a table below the query.

```
678 WITH product_return_summary AS (  
679     SELECT  
680         p.product_id,  
681         p.product_name,  
682  
683         SUM(foi.quantity) AS total_quantity_sold,  
684  
685         COALESCE(SUM(fr.quantity_returned), 0) AS total_quantity_returned  
686  
687     FROM dim_product p  
688  
689     JOIN fact_order_items foi  
690         ON p.product_id = foi.product_id  
691  
692     LEFT JOIN fact_returns fr  
693         ON foi.order_item_id = fr.order_item_id  
694 )  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

	product_id [PK] integer	product_name character varying (150)	total_quantity_sold bigint	total_quantity_returned bigint	return_rate numeric
1	101	Kitchen Storage Set 101	100	50	50.00
2	89	Cotton T-Shirt 89	100	50	50.00
3	104	Car Phone Holder 104	200	50	25.00
4	59	Cotton T-Shirt 59	201	34	16.92
5	69	Cotton T-Shirt 69	199	33	16.58
6	79	Cotton T-Shirt 79	200	33	16.50
7	71	Kitchen Storage Set 71	100	17	17.00

Section 4 — Views

Q15. Customer Sales Summary View

Create a view:

```
vw_customer_sales_summary
```

It should contain:

- Customer ID
- Customer Name
- Region

- Customer Segment
- Total Orders
- Total Items Purchased
- Total Spending
- Average Order Value

```
CREATE VIEW vw_customer_sales_summary AS

SELECT
    c.customer_id,
    c.customer_name,
    r.region_zone AS region,
    c.customer_segment,

    COUNT(DISTINCT o.order_id) AS total_orders,

    SUM(foi.quantity) AS total_items_purchased,

    SUM(o.final_order_amount) AS total_spending,

    AVG(o.final_order_amount) AS average_order_value

FROM dim_customer c

JOIN dim_region r
    ON c.region_id = r.region_id

JOIN fact_orders o
    ON c.customer_id = o.customer_id

JOIN fact_order_items foi
    ON o.order_id = foi.order_id
```

GROUP BY

```
c.customer_id,  
c.customer_name,  
r.region_zone,  
c.customer_segment;
```

SELECT *

FROM vw_customer_sales_summary;

The screenshot shows a SQL IDE interface. On the left is a sidebar with a tree view of database objects including Sequences, Tables (16), Trigger Functions, Types, Views, Subscriptions, and Roles. The main area is divided into 'Query' and 'Data Output' tabs. The 'Query' tab shows a SQL script that creates a view named 'vw_customer_sales_summary' and then selects data from it. The 'Data Output' tab shows the results of the query as a table with 8 columns and 6 rows of data.

```
CREATE VIEW vw_customer_sales_summary AS  
  
SELECT  
    c.customer_id,  
    c.customer_name,  
    r.region_zone AS region,  
    c.customer_segment,  
  
    COUNT(DISTINCT o.order_id) AS total_orders,  
  
    SUM(oi.quantity) AS total_items_purchased,  
  
    SUM(o.final_order_amount) AS total_spending,  
  
    AVG(o.final_order_amount) AS average_order_value  
  
FROM dim_customer c
```

	customer_id	customer_name	region	customer_segment	total_orders	total_items_purchased	total_spending	average_order_value
1	1	Aarav Gupta 1	West	Premium	15	15	519882.00	34658.800000000000
2	2	Vivaan Iyer 2	West	Premium	7	7	414168.62	59166.945714285714
3	3	Aditya Joshi 3	West	Premium	7	7	141408.00	20201.142857142857
4	4	Arjun Singh 4	West	Premium	8	8	67241.80	8405.2250000000000000
5	5	Rohan Shah 5	West	Premium	7	7	199870.92	28552.988571428571
6	6	Kunal More 6	West	Premium	8	8	376774.40	47096.800000000000

Q16. Product Performance View

Create:

vw_product_sales_performance

Include:

- Product
- SKU
- Category

- Brand
- Units Sold
- Gross Sales
- Discounts
- Net Sales
- Returned Quantity
- Return Rate

```
CREATE VIEW vw_product_sales_performance AS

SELECT
    p.product_name AS product,
    p.sku_code AS sku,
    c.category_name AS category,
    p.brand_name AS brand,

    SUM(foi.quantity) AS units_sold,

    SUM(foi.quantity * foi.unit_price) AS gross_sales,

    SUM(foi.item_discount) AS discounts,

    SUM(foi.item_total_amount) AS net_sales,

    COALESCE(SUM(fr.quantity_returned), 0) AS returned_quantit
ty,

    ROUND(
        COALESCE(SUM(fr.quantity_returned), 0) * 100.0
        / NULLIF(SUM(foi.quantity), 0),
        2
    ) AS return_rate
```

```
FROM dim_product p

JOIN dim_category c
  ON p.category_id = c.category_id

JOIN fact_order_items foi
  ON p.product_id = foi.product_id

LEFT JOIN fact_returns fr
  ON foi.order_item_id = fr.order_item_id

GROUP BY
  p.product_id,
  p.product_name,
  p.sku_code,
  c.category_name,
  p.brand_name;

SELECT *
FROM vw_product_sales_performance;
```


dim_calendar

dim_category

dim_customer

dim_delivery_partner

dim_product

dim_region

dim_seller

dim_warehouse

fact_customer_reviews

fact_inventory_snapshot

fact_order_items

fact_orders

fact_payments

fact_returns

fact_seller_performance

fact_shipments

Trigger Functions

Types

Views (2)

vw_customer_sales_sumi

vw_product_sales_perfor

public

Subscriptions

ymmerce_order_delivery_db

stgres

/Group Roles

spaces

Query

Query History

836

ON p.product_id = foi.product_id

837

838

LEFT JOIN fact_returns fr

839

ON foi.order_item_id = fr.order_item_id

840

841

GROUP BY

842

p.product_id,

843

p.product_name,

844

p.sku_code,

845

c.category_name,

846

p.brand_name;

847

848

849

SELECT *

850

FROM vw_product_sales_performance;

Data Output

Messages

Notifications

Showing rows: 1 to 100

Page No: 1 of 1

	product character varying (150)	sku character varying (30)	category character varying (100)	brand character varying (100)	units_sold bigint	gross_sales numeric	discounts numeric	net_sales numeric	returned_quantity bigint	return_rate numeric
1	Galaxy A Series Smartphone 1	SKU-00001	Mobiles & Smartphones	Dell	1072	21929154.94	1270197.39	20658957.55	65	6.06
2	Redmi Note Smartphone 2	SKU-00002	Mobiles & Smartphones	Lakme	1339	45430030.14	1378446.68	44051583.46	85	6.35
3	OnePlus Nord Smartphone 3	SKU-00003	Mobiles & Smartphones	Samsung	1262	69588767.43	2562458.70	57026308.73	67	5.31
4	iPhone Series Smartphone 4	SKU-00004	Mobiles & Smartphones	LG	1394	82802319.24	3144935.56	79657383.68	67	4.81
5	Pixel Series Smartphone 5	SKU-00005	Mobiles & Smartphones	Philips	1139	82186400.90	3506069.80	78680331.10	65	5.71
6	Galaxy A Series Smartphone 6	SKU-00006	Mobiles & Smartphones	Apple	1069	91070870.68	5265082.92	85805787.76	68	6.36
7	Redmi Note Smartphone 7	SKU-00007	Mobiles & Smartphones	Sony	1327	24722119.80	751393.02	23970726.78	83	6.25
8	OnePlus Nord Smartphone 8	SKU-00008	Mobiles & Smartphones	Prestige	1265	40316881.60	1727723.52	38589158.08	66	5.22

Q17. Delivery Performance View

Create:

`vw_delivery_performance`

Include:

- Delivery Partner
- Total Shipments
- Delivered Shipments
- Delayed Shipments
- Average Delivery Days
- Average Delay Days
- Shipping Cost
- On-Time Delivery %

CREATE VIEW vw_delivery_performance **AS**

SELECT

```

dp.partner_name AS delivery_partner,

COUNT(fs.shipment_id) AS total_shipments,

COUNT(CASE
    WHEN fs.actual_delivery_date IS NOT NULL
    THEN 1
END )
AS delivered_shipments,

COUNT(CASE
    WHEN fs.actual_delivery_date IS NOT NULL
    AND fs.actual_delivery_date > fs.estimated_d
elivery_date
    THEN 1
END )
AS delayed_shipments,

ROUND(AVG(CASE
    WHEN fs.actual_delivery_date IS NOT NULL
    THEN fs.actual_delivery_date - fs.shipment_da
te
    END ),2)
AS average_delivery_days,

ROUND(
    AVG(CASE
        WHEN fs.actual_delivery_date IS NOT NULL
        AND fs.actual_delivery_date > fs.estimat
ed_delivery_date
        THEN fs.actual_delivery_date
        - fs.estimated_delivery_date
    END),2)
AS average_delay_days,

SUM(fs.shipping_cost) AS shipping_cost,

```

```

        ROUND(COUNT(CASE
                        WHEN fs.actual_delivery_date IS NOT N
ULL
                        AND fs.actual_delivery_date <= f
s.estimated_delivery_date
                        THEN 1
                        END ) * 100.0 / NULLIF(
                        COUNT(
                            CASE
                                WHEN fs.actual_delivery_date
IS NOT NULL
                                THEN 1
                                END ),0),2) AS on_time_delivery_p
ercentage

FROM fact_shipments fs

JOIN dim_delivery_partner dp
    ON fs.delivery_partner_id = dp.delivery_partner_id

GROUP BY
    dp.delivery_partner_id,
    dp.partner_name;

SELECT *
FROM vw_delivery_performance

```

dim_category

dim_customer

dim_delivery_partner

dim_product

dim_region

dim_seller

dim_warehouse

fact_customer_reviews

fact_inventory_snapshot

fact_order_items

fact_orders

fact_payments

fact_returns

fact_seller_performance

fact_shipments

Trigger Functions

Types

Views (3)

vw_customer_sales_summary

vw_delivery_performance

vw_product_sales_performance

public

Subscriptions

commerce_order_delivery_db

postgres

Group Roles

Query History

926) AS on_time_delivery_percentage

927

928 FROM fact_shipments fs

929

930 JOIN dim_delivery_partner dp

931 ON fs.delivery_partner_id = dp.delivery_partner_id

932

933 GROUP BY

934 dp.delivery_partner_id,

935 dp.partner_name;

936

937

938 SELECT *

939 FROM vw_delivery_performance

940

941

942

Data Output

Messages

Notifications

Showing rows: 1 to 7

Page No: 1 of 1

	delivery_partner character varying (100)	total_shipments bigint	delivered_shipments bigint	delayed_shipments bigint	average_delivery_days numeric	average_delay_days numeric	shipping_cost numeric	on_time_delivery_percentage numeric
1	Delhivery	3800	3800	0	2.79	[null]	1488069.05	100.00
2	XpressBees	2200	2200	1000	4.45	2.00	909562.50	54.55
3	Ecom Express	3000	3000	0	3.47	[null]	1192173.40	100.00
4	Shadowfax	1000	1000	1000	6.20	2.80	407304.30	0.00
5	BlueDart Logistics	4000	4000	0	2.57	[null]	1450785.20	100.00
6	Regional Courier	400	400	400	10.99	7.50	168345.00	0.00
7	FastTrack Logistics	4800	2800	0	2.57	[null]	1702100.60	100.00

Q18. View-Based Customer Analysis

Using `vw_customer_sales_summary`, find the **Top 20 customers by total spending**.

Display:

- Customer
- Region
- Segment
- Total Orders
- Total Spending
- Average Order Value

Do not directly query the underlying customer/order tables for this question.

```

SELECT
customer_name AS customer,
region,
customer_segment AS segment,

```

```

total_orders,
total_spending,
average_order_value
FROM vw_customer_sales_summary
ORDER BY total_spending DESC
LIMIT 30;

```

The screenshot shows a database management interface. On the left is a sidebar with a tree view of database objects. The 'vw_customer_sales_summary' view is selected, showing its columns: customer_id, customer_name, region, customer_segment, total_orders, total_items_purchased, total_spending, and average_order_value. The main panel is divided into 'Query' and 'Data Output' sections. The 'Query' section displays the SQL query from the previous block. The 'Data Output' section shows the results of the query as a table with 10 rows.

	customer	region	segment	total_orders	total_spending	average_order_value
	character varying (100)	character varying (50)	character varying (50)	bigint	numeric	numeric
1	Aditi Sharma 2428	West	Affluent	28	35812646.16	426341.025714285714
2	Meera Joshi 2658	West	Mass Affluent	29	34262420.80	295365.696551724138
3	Kavya Kulkarni 2514	West	Affluent	28	34016934.66	404963.507857142857
4	Kavya Singh 2314	West	Affluent	29	29398415.88	337912.826206896552
5	Meera Nair 2858	West	Mass Affluent	28	29034028.00	259232.392857142857
6	Meera Joshi 2358	West	Mass Affluent	29	27737281.60	239114.496551724138
7	Aditi Joshi 2628	West	Affluent	29	26810502.42	308166.694482758621
8	Meera Nair 2558	West	Mass Affluent	29	26466962.40	228163.468965517241
9	Neha Iyer 2372	West	Mass Market	29	25599623.20	220686.406896551724
10	Neha Deshmukh 2472	West	Mass Market	29	25385570.40	218841.124137931034

Section 5 — PostgreSQL Functions

Q19. Customer Spending Function

Create:

```
fn_customer_total_spending(customer_id)
```

The function should return the customer's:

- Total Orders

- Total Spending

Test the function for at least **5 customers**.

```
CREATE OR REPLACE FUNCTION fn_customer_total_spending(p_customer_id INT)
RETURNS TABLE (
    total_orders BIGINT,
    total_spending DECIMAL(10,2)
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(o.order_id),
        COALESCE(SUM(o.final_order_amount), 0)
    FROM fact_orders o
    WHERE o.customer_id = p_customer_id;
END;
$$;
```

```
SELECT
    1 AS customer_id,
    *
FROM fn_customer_total_spending(1)

UNION ALL

SELECT
    2 AS customer_id,
    *
FROM fn_customer_total_spending(2)
```

UNION ALL

SELECT

3 AS customer_id,

*

FROM fn_customer_total_spending(3)

UNION ALL

SELECT

4 AS customer_id,

*

FROM fn_customer_total_spending(4)

UNION ALL

SELECT

5 AS customer_id,

*

FROM fn_customer_total_spending(5);

The screenshot shows a SQL IDE interface. On the left is a tree view of database objects, including 'Functions (1)' with 'fn_customer_total_spending(p_cus'. The main area displays a SQL query in the 'Query' tab, with line numbers 997 to 1015. The query calculates total orders and spending for each customer. Below the query is the 'Data Output' tab, which shows a table with 5 rows of results.

```

997      COUNT(o.order_id),
998      COALESCE(SUM(o.final_order_amount), 0)
999      FROM fact_orders o
1000     WHERE o.customer_id = p_customer_id;
1001 END;
1002 $$;
1003
1004 |
1005
1006 SELECT
1007     1 AS customer_id,
1008     *
1009 FROM fn_customer_total_spending(1)
1010
1011 UNION ALL
1012
1013 SELECT
1014     2 AS customer_id,
1015     *

```

	customer_id integer	total_orders bigint	total_spending numeric
1	1	15	519882.00
2	2	7	414168.62
3	3	7	141408.00
4	4	8	67241.80
5	5	7	199870.92

Q20. Order Value Classification Function

Create:

```
fn_order_value_category(order_amount)
```

The function should return:

Amount	Category
< ₹1,000	Small
₹1,000–₹5,000	Medium
₹5,000–₹20,000	Large
> ₹20,000	Premium

Test the function with multiple order amounts.


```

CREATE OR REPLACE FUNCTION fn_order_value_category( p_order_a
mount DECIMAL(10,2))
RETURNS VARCHAR(20)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN CASE
        WHEN p_order_amount < 1000 THEN 'Small'
        WHEN p_order_amount <= 5000 THEN 'Medium'
        WHEN p_order_amount <= 20000 THEN 'Large'
        ELSE 'Premium'
    END;
END;
$$;

```

-- Test Data :

```

SELECT
    amount,
    fn_order_value_category(amount) AS order_value_category
FROM (
    VALUES
        (800::DECIMAL),
        (1000::DECIMAL),
        (3500::DECIMAL),
        (5000::DECIMAL),
        (10000::DECIMAL),
        (20000::DECIMAL),
        (25000::DECIMAL)
) AS orders(amount);

```

The screenshot shows a database IDE with a sidebar on the left containing a tree view of database objects. The main area is divided into two panes: 'Query' and 'Query History'. The 'Query' pane displays a SQL script for creating a function named `fn_order_value_category`. The 'Query History' pane shows the execution of this query. Below the query panes, there is a 'Data Output' pane showing the results of the query. The results are displayed in a table with two columns: 'amount' and 'order_value_category'.

```

1059
1060 CREATE OR REPLACE FUNCTION fn_order_value_category( p_order_amount DECIMAL(10,2))
1061 RETURNS VARCHAR(20)
1062 LANGUAGE plpgsql
1063 AS $$
1064 BEGIN
1065     RETURN CASE
1066         WHEN p_order_amount < 1000 THEN 'Small'
1067         WHEN p_order_amount <= 5000 THEN 'Medium'
1068         WHEN p_order_amount <= 20000 THEN 'Large'
1069         ELSE 'Premium'
1070     END;
1071 END;
1072 $$;
1073
1074
1075 -- Test Data :
1076
1077 SELECT

```

	amount numeric	order_value_category character varying
1	800	Small
2	1000	Medium
3	3500	Medium
4	5000	Medium
5	10000	Large
6	20000	Large

Q21. Delivery Delay Function

Create:

```
fn_delivery_delay_category(estimated_date, actual_date)
```

The function should return:

- Delivered Early
- On Time
- 1–3 Days Late
- 4–7 Days Late
- More Than 7 Days Late
- Not Delivered

Test the function with different date scenarios.

```

CREATE OR REPLACE FUNCTION fn_delivery_delay_category(
    p_estimated_date DATE,
    p_actual_date DATE
)
RETURNS VARCHAR(30)
LANGUAGE plpgsql
AS $$
DECLARE
    delay_days INT;
BEGIN

    IF p_actual_date IS NULL THEN
        RETURN 'Not Delivered';
    END IF;

    delay_days := p_actual_date - p_estimated_date;

    RETURN CASE
        WHEN delay_days < 0 THEN 'Delivered Early'
        WHEN delay_days = 0 THEN 'On Time'
        WHEN delay_days BETWEEN 1 AND 3 THEN '1-3 Days Late'
        WHEN delay_days BETWEEN 4 AND 7 THEN '4-7 Days Late'
        ELSE 'More Than 7 Days Late'
    END;

END;
$$;

SELECT fn_delivery_delay_category(
    '2026-08-10',
    '2026-08-08'
);

SELECT fn_delivery_delay_category(

```

```
'2026-08-10',
'2026-08-18'
);
```

The screenshot shows a database IDE with a left sidebar containing a tree view of database objects. The 'Functions (3)' folder is expanded, showing three functions: `fn_customer_total_spending(p...`, `fn_delivery_delay_category(p...`, and `fn_order_value_category(p...`. The main window displays a SQL query in a dark theme, with line numbers 1119 to 1142. The query is a PL/SQL function definition for `fn_delivery_delay_category`. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with one row: `fn_delivery_delay_category` with a value of 'More Than 7 Days Late'.

```
1119 LANGUAGE plpgsql
1120 AS $$
1121 DECLARE
1122     delay_days INT;
1123 BEGIN
1124     -- If actual delivery date is NULL
1125     IF p_actual_date IS NULL THEN
1126         RETURN 'Not Delivered';
1127     END IF;
1128
1129     -- Calculate difference between actual and estimated date
1130     delay_days := p_actual_date - p_estimated_date;
1131
1132     RETURN CASE
1133         WHEN delay_days < 0 THEN 'Delivered Early'
1134         WHEN delay_days = 0 THEN 'On Time'
1135         WHEN delay_days BETWEEN 1 AND 3 THEN '1-3 Days Late'
1136         WHEN delay_days BETWEEN 4 AND 7 THEN '4-7 Days Late'
1137         ELSE 'More Than 7 Days Late'
1138     END;
1139 END;
1140 $$;
```

fn_delivery_delay_category
More Than 7 Days Late

Section 6 — Stored Procedures

Q22. Process Order Return Procedure

Create:

```
sp_process_order_return()
```

Parameters should include:

- Order Item ID

- Return Date
- Return Reason
- Quantity Returned
- Refund Amount

The procedure should:

1. Validate that the order item exists.
2. Validate that return quantity does not exceed ordered quantity.
3. Insert the return record.
4. Set an appropriate return status.

```
CREATE OR REPLACE PROCEDURE sp_process_order_return(
    p_order_item_id INT,
    p_return_date DATE,
    p_return_reason VARCHAR(100),
    p_quantity_returned INT,
    p_refund_amount DECIMAL(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_ordered_quantity INT;
BEGIN

    -- 1. Validate that order item exists
    SELECT quantity
    INTO v_ordered_quantity
    FROM fact_order_items
    WHERE order_item_id = p_order_item_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION
```

```

        'Order item ID % does not exist.',
        p_order_item_id;
END IF;

-- Validate return quantity
IF p_quantity_returned <= 0 THEN
    RAISE EXCEPTION
        'Return quantity must be greater than 0.';
END IF;

IF p_quantity_returned > v_ordered_quantity THEN
    RAISE EXCEPTION
        'Return quantity (%) cannot exceed ordered quantity (%)',
        p_quantity_returned,
        v_ordered_quantity;
END IF;

INSERT INTO fact_returns (
    order_item_id,
    return_date,
    return_reason,
    quantity_returned,
    refund_amount,
    return_status
)
VALUES (
    p_order_item_id,
    p_return_date,
    p_return_reason,
    p_quantity_returned,
    p_refund_amount,
    'Approved'

```

```

);

RAISE NOTICE
    'Return processed successfully for Order Item ID %.',
    p_order_item_id;

END;
$$;

CALL sp_process_order_return(
    1,
    '2026-08-21',
    'Damaged Product',
    1,
    500.00
);

SELECT *
FROM fact_returns
WHERE order_item_id = 1;

```

The screenshot shows a SQL IDE with a left sidebar containing a database schema tree. The main window displays a SQL script for a stored procedure named `sp_process_order_return`. The script includes a confirmation message, a `RAISE NOTICE` statement, and a `CALL` statement. Below the script, the 'Data Output' tab shows the results of the procedure execution, which is a table with 8 columns and 2 rows.

```

1244 );
1245
1246
1247 -- 4. Confirmation message
1248 RAISE NOTICE
1249 'Return processed successfully for Order Item ID %.',
1250 p_order_item_id;
1251
1252 END;
1253 $$;
1254
1255
1256
1257 CALL sp_process_order_return(
1258 1,
1259 '2026-08-21',
1260 'Damaged Product',
1261 1,
1262 500.00
1263 );
1264
1265 SELECT *
1266 FROM fact_returns

```

	return_id [PK] integer	order_item_id integer	return_date date	return_reason character varying (100)	quantity_returned integer	refund_amount numeric (10,2)	return_status character varying (30)
1	3001	1	2026-08-21	Damaged Product	1	500.00	Approved
2	3002	1	2026-08-21	Damaged Product	1	500.00	Approved

Total rows: 2 Query complete 00:00:00.199

Q23. Process Customer Review Procedure

Create:

```
sp_add_customer_review()
```

Parameters:

- Order Item ID
- Customer ID
- Review Date
- Rating
- Review Comment

The procedure should:

1. Validate that the order item exists.
2. Validate the rating is between **1 and 5**.
3. Insert the review.

4. Automatically determine `verified_purchase_flag` based on whether the customer actually purchased the item.

```
CREATE OR REPLACE PROCEDURE sp_add_customer_review(  
    p_order_item_id INT,  
    p_customer_id INT,  
    p_review_date DATE,  
    p_rating DECIMAL(2,1),  
    p_review_comment TEXT  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_order_exists BOOLEAN;  
    v_customer_purchased BOOLEAN;  
BEGIN  
  
    SELECT EXISTS (  
        SELECT 1  
        FROM fact_order_items  
        WHERE order_item_id = p_order_item_id  
    )  
    INTO v_order_exists;  
  
    IF NOT v_order_exists THEN  
        RAISE EXCEPTION  
            'Order Item ID % does not exist.',  
            p_order_item_id;  
    END IF;  
  
    IF p_rating < 1 OR p_rating > 5 THEN  
        RAISE EXCEPTION  
            'Rating must be between 1 and 5.';
```

```

END IF;

SELECT EXISTS (
    SELECT 1
    FROM fact_order_items foi
    JOIN fact_orders fo
        ON foi.order_id = fo.order_id
    WHERE foi.order_item_id = p_order_item_id
        AND fo.customer_id = p_customer_id
)
INTO v_customer_purchased;

INSERT INTO fact_customer_reviews (
    order_item_id,
    customer_id,
    review_date,
    rating,
    review_comment,
    verified_purchase_flag
)
VALUES (
    p_order_item_id,
    p_customer_id,
    p_review_date,
    p_rating,
    p_review_comment,
    v_customer_purchased
);

RAISE NOTICE
    'Customer review added successfully for Order Item ID
    %, ',
    p_order_item_id;

```

```

END;
$$;

CALL sp_add_customer_review(
    101,
    5,
    '2026-08-25',
    4.5,
    'Good product'
);

SELECT *
FROM fact_customer_reviews
ORDER BY review_id DESC;

```

The screenshot shows a database management interface. On the left is a sidebar with a tree view of database objects. The main area displays SQL code with line numbers 1367 to 1380. Below the code is a 'Data Output' tab showing a table of customer reviews. The table has 8 columns: review_id, order_item_id, customer_id, review_date, rating, review_comment, and verified_purchase_flag. It displays 9 rows of data, sorted by review_id in descending order.

review_id	order_item_id	customer_id	review_date	rating	review_comment	verified_purchase_flag
1	9551	101	2026-08-25	4.5	Good product	false
2	9550	34929	2024-12-03	4.0	Good product and delivery experien...	true
3	9549	34928	2024-11-25	4.0	Good product and delivery experien...	true
4	9548	34927	2024-11-22	3.0	Average experience	true
5	9547	34926	2024-11-07	3.0	Average experience	true
6	9546	34925	2024-11-04	3.0	Average experience	true
7	9545	34924	2024-11-11	3.0	Average experience	true
8	9544	34923	2024-11-08	3.0	Average experience	true
9	9543	34922	2024-10-31	3.0	Average experience	true

Q24. Shipment Status Processing Procedure

Create:

```
sp_update_shipment_status()
```

The procedure should update shipment status based on the available shipment dates.

Suggested business rules:

```
Actual Delivery Date exists
    → Delivered

Shipment Date exists but Actual Delivery Date is NULL
    → In Transit / Shipped

No Actual Delivery + Expected Date passed
    → Delayed
```

Use appropriate `CASE` logic and date comparison.

```
CREATE OR REPLACE PROCEDURE sp_update_shipment_status()
LANGUAGE plpgsql
AS $$
BEGIN

    UPDATE fact_shipments
    SET shipment_status =
        CASE

            WHEN actual_delivery_date IS NOT NULL
            THEN 'Delivered'

            WHEN actual_delivery_date IS NULL
            AND estimated_delivery_date < CURRENT_DATE
            THEN 'Delayed'

            WHEN shipment_date IS NOT NULL
            AND actual_delivery_date IS NULL
```

```
        THEN 'In Transit'

        ELSE 'Shipped'

    END;

    RAISE NOTICE 'Shipment statuses updated successfully.';

END;
$$;

CALL sp_update_shipment_status();

SELECT
    shipment_id,
    shipment_date,
    estimated_delivery_date,
    actual_delivery_date,
    shipment_status
FROM fact_shipments
ORDER BY shipment_id;
```

```

1439 END;
1440 $$;
1441
1442
1443
1444 CALL sp_update_shipment_status();
1445
1446
1447 SELECT
1448     shipment_id,
1449     shipment_date,
1450     estimated_delivery_date,
1451     actual_delivery_date,
1452     shipment_status
1453 FROM fact_shipments
1454 ORDER BY shipment_id;
1455

```

	shipment_id [PK] integer	shipment_date date	estimated_delivery_date date	actual_delivery_date date	shipment_status character varying (30)
1	1	2024-01-30	2024-02-06	[null]	Delayed
2	2	2024-02-07	2024-02-11	[null]	Delayed
3	3	2024-02-13	2024-02-16	[null]	Delayed
4	4	2024-02-21	2024-02-21	[null]	Delayed
5	5	2024-02-27	2024-03-04	[null]	Delayed
6	6	2024-03-06	2024-03-09	[null]	Delayed
7	7	2024-03-12	2024-03-14	[null]	Delayed

Section 7 — Final Business Challenge

Q25. Executive E-Commerce Performance Report ★

Create one management-level report showing **performance of each product category**.

The final output should contain:

- Category
- Total Orders
- Units Sold
- Gross Sales
- Total Discount
- Net Sales

- Average Order Value
- Returned Units
- Return Rate
- Delivered Orders
- Delayed Orders
- On-Time Delivery %
- Average Customer Rating

Requirements

Your solution **must use**:

-  At least **2 CTEs**
-  Multiple JOINS
-  `CASE`
-  Date functions
-  Aggregate functions
-  Meaningful aliases

```
WITH sales_summary AS (

    SELECT
        c.category_id,
        c.category_name AS category,

        COUNT(DISTINCT foi.order_id) AS total_orders,

        SUM(foi.quantity) AS units_sold,

        SUM(foi.quantity * foi.unit_price) AS gross_sales,
```

```

        SUM(foi.item_discount) AS total_discount,

        SUM(foi.item_total_amount) AS net_sales,

        AVG(o.final_order_amount) AS average_order_value

FROM dim_category c

JOIN dim_product p
    ON c.category_id = p.category_id

JOIN fact_order_items foi
    ON p.product_id = foi.product_id

JOIN fact_orders o
    ON foi.order_id = o.order_id

GROUP BY
    c.category_id,
    c.category_name
),

return_summary AS (

    SELECT
        c.category_id,

        COALESCE(
            SUM(fr.quantity_returned),
            0
        ) AS returned_units

FROM dim_category c

JOIN dim_product p
    ON c.category_id = p.category_id

```



```

JOIN fact_order_items foi
    ON p.product_id = foi.product_id

LEFT JOIN fact_returns fr
    ON foi.order_item_id = fr.order_item_id

GROUP BY
    c.category_id
),

delivery_summary AS (

    SELECT
        c.category_id,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                THEN fs.order_id
            END
        ) AS delivered_orders,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NULL
                AND fs.estimated_delivery_date < CURRENT
_DATE
                THEN fs.order_id
            END
        ) AS delayed_orders,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                AND fs.actual_delivery_date

```

```

                <= fs.estimated_delivery_date
            THEN fs.order_id
        END
    ) AS on_time_orders

FROM dim_category c

JOIN dim_product p
    ON c.category_id = p.category_id

JOIN fact_order_items foi
    ON p.product_id = foi.product_id

JOIN fact_shipments fs
    ON foi.order_id = fs.order_id

GROUP BY
    c.category_id
),

rating_summary AS (

    SELECT
        c.category_id,

        AVG(fcr.rating) AS average_customer_rating

    FROM dim_category c

    JOIN dim_product p
        ON c.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    JOIN fact_customer_reviews fcr

```

```

        ON foi.order_item_id = fcr.order_item_id

    GROUP BY
        c.category_id
)

SELECT
    ss.category,

    ss.total_orders,

    ss.units_sold,

    ROUND(ss.gross_sales, 2) AS gross_sales,

    ROUND(ss.total_discount, 2) AS total_discount,

    ROUND(ss.net_sales, 2) AS net_sales,

    ROUND(ss.average_order_value, 2) AS average_order_value,

    rs.returned_units,

    ROUND(
        rs.returned_units * 100.0
        / NULLIF(ss.units_sold, 0),
        2
    ) AS return_rate,

    COALESCE(ds.delivered_orders, 0) AS delivered_orders,

    COALESCE(ds.delayed_orders, 0) AS delayed_orders,

    ROUND(
        COALESCE(ds.on_time_orders, 0) * 100.0
        / NULLIF(

```

```

        COALESCE(ds.delivered_orders, 0),
        0
    ),
    2
) AS on_time_delivery_percentage,

ROUND(
    COALESCE(rts.average_customer_rating, 0),
    2
) AS average_customer_rating

FROM sales_summary ss

LEFT JOIN return_summary rs
    ON ss.category_id = rs.category_id

LEFT JOIN delivery_summary ds
    ON ss.category_id = ds.category_id

LEFT JOIN rating_summary rts
    ON ss.category_id = rts.category_id

ORDER BY
    ss.net_sales DESC;

```

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions (3)

fn_customer_total_sp

fn_delivery_delay_cat

fn_order_value_cat

Materialized Views

Operators

Procedures (3)

sp_add_customer_re

sp_process_order_re

sp_update_shipment

Sequences

Tables

Trigger Functions

Types

Views (3)

vw_customer_sales_

vw_delivery_perform

vw_product_sales_p

public

Subscriptions

Query History

WITH sales_summary AS (

SELECT

c.category_id,

c.category_name AS category,

COUNT(DISTINCT foi.order_id) AS total_orders,

SUM(foi.quantity) AS units_sold,

SUM(foi.quantity * foi.unit_price) AS gross_sales,

SUM(foi.item_discount) AS total_discount,

SUM(foi.item_total_amount) AS net_sales,

AVG(foi.final_order_amount) AS average_order_value

Data Output

Messages

Notifications

Showing rows: 1 to 16

Page No: 1 of 1

	category character varying (100)	total_orders bigint	units_sold bigint	gross_sales numeric	total_discount numeric	net_sales numeric	average_order_value numeric	returned_units bigint	return_rate numeric	delivered_orders bigint	delayed_orders bigint	on_time number
1	Mobiles & Smartphones	11000	18600	871322266.25	36957765.15	834364501.10	112658.72	1052	5.66	9800	400	
2	Laptops	3600	4887	423237573.44	16755415.67	406482157.77	154167.41	473	9.68	3256	344	
3	Televisions	3600	4888	372706069.76	14904959.41	357801110.35	139190.23	470	9.62	3256	344	
4	Furniture	1299	1499	55975038.96	1875338.52	54099700.44	104018.99	117	7.81	1299	0	
5	Home Appliances	2001	2602	34346774.81	1307188.35	33039586.46	68980.93	233	8.95	1801	200	
6	Kitchen Appliances	2000	2600	25519852.98	960664.34	24559188.64	61912.52	234	9.00	1800	200	